Homework 7

When Your Startup's Flash Sale Almost Failed

CS 6650 - Distributed Systems

Mayank Tamakuwala

Part II: Synchronous vs Asynchronous Order Processing

Infrastructure Overview

All infrastructure was provisioned using Terraform in the us-west-2 region.

Resource	Configuration
VPC	10.0.0.0/16
Public Subnets (ALB)	10.0.1.0/24, 10.0.2.0/24
Private Subnets (ECS)	10.0.10.0/24, 10.0.11.0/24
ECS Task Size	256 CPU, 512 MB (Fargate)
SNS Topic	hw7-orders
SQS Queue	hw7-orders (vis 60s, retention 4d)
Payment Delay	3 seconds (buffered channel)
Sync Concurrency	15 buffered channel slots

Application Architecture

A single Go binary runs in two modes controlled by the APP_MODE environment variable:
- Receiver (ECS service behind ALB): exposes POST /orders/sync and POST /orders/async
- Processor (ECS service): polls SQS and processes orders with configurable worker goroutines

Synchronous flow:

```
Customer -> API -> Payment (3s buffered channel) -> 200 OK
```

Asynchronous flow:

```
Customer -> API -> SNS -> 202 Accepted (instant)
                    |
               SQS Queue
                    |
            ECS Workers -> Payment (3s) -> Delete message
```

Phase 1: Sync Normal Operations

Test config: 5 concurrent users, 30 seconds, spawn rate 1 user/sec, wait time 100-500ms

Metric	Value
Total Requests	33
Failure Rate	0%
Avg Response Time	3,168 ms
Median	3,200 ms
P99	3,400 ms
Throughput	1.25 req/s

With only 5 users and 15 available concurrency slots, every request gets immediate access to a payment slot. Each request waits the full 3-second payment delay and returns successfully. The system handles normal load without any contention.

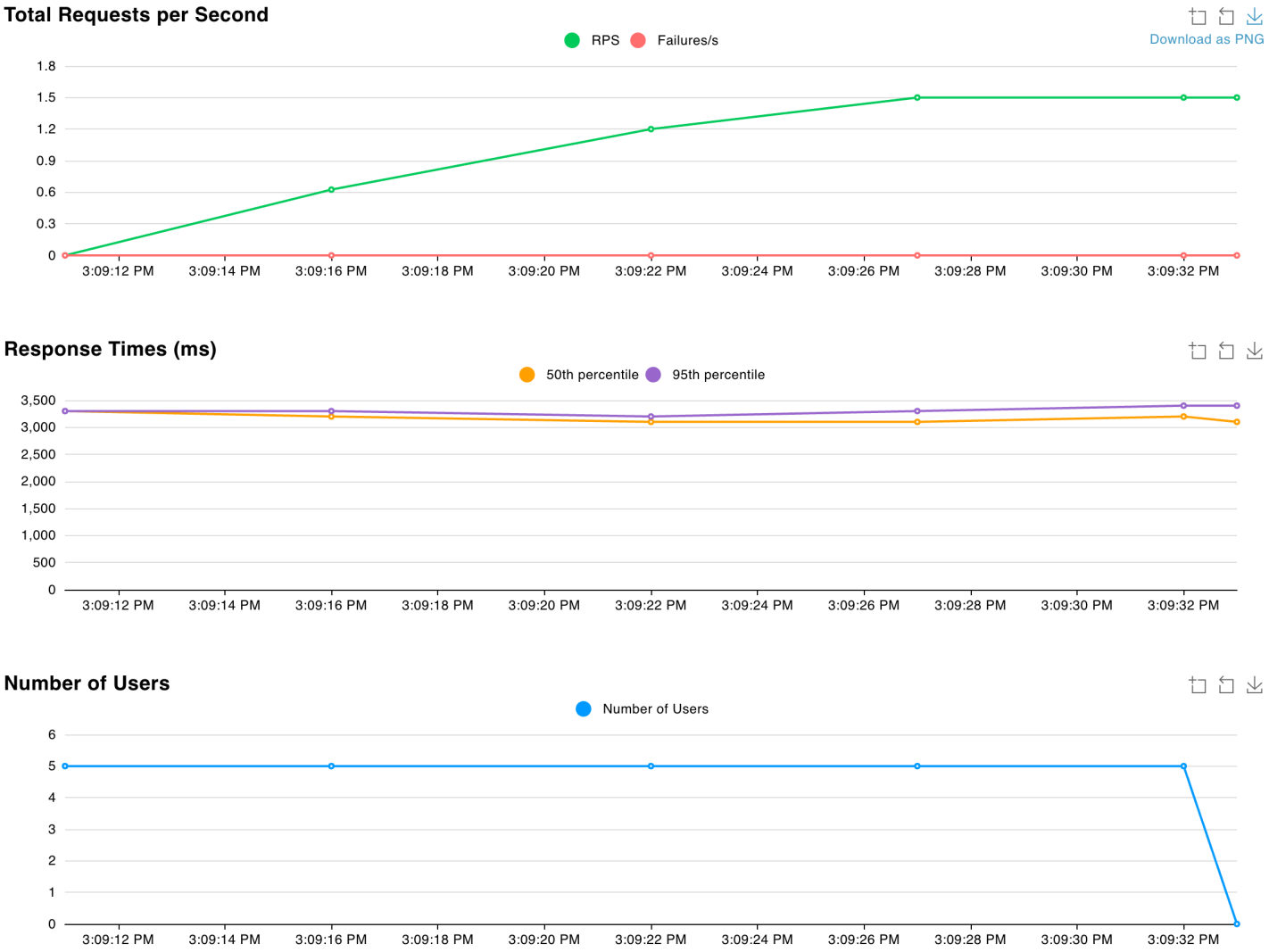

Figure: Sync Normal - stable ~3.2s response time with 5 users

Phase 2: Sync Flash Sale - The Bottleneck

Test config: 20 concurrent users, 60 seconds, spawn rate 10 users/sec, wait time 100-500ms

Metric	Value
Total Requests	285
Failure Rate	0%
Avg Response Time	3,720 ms
P95	4,900 ms
P99	5,500 ms
Throughput	4.85 req/s

Bottleneck analysis:

- Payment processor speed: 1 order per 3 seconds per slot
- With 15 concurrency slots: max throughput = 15 / 3 = 5 orders/second
- With 20 concurrent users: demand exceeds capacity
- Result: requests queue behind the buffered channel, latency spikes to 5.5s

While 0% of requests failed (the buffered channel absorbs overflow), customers experience response times nearly double the baseline. At true flash sale scale (60 orders/sec), the system would see timeouts and lost sales.

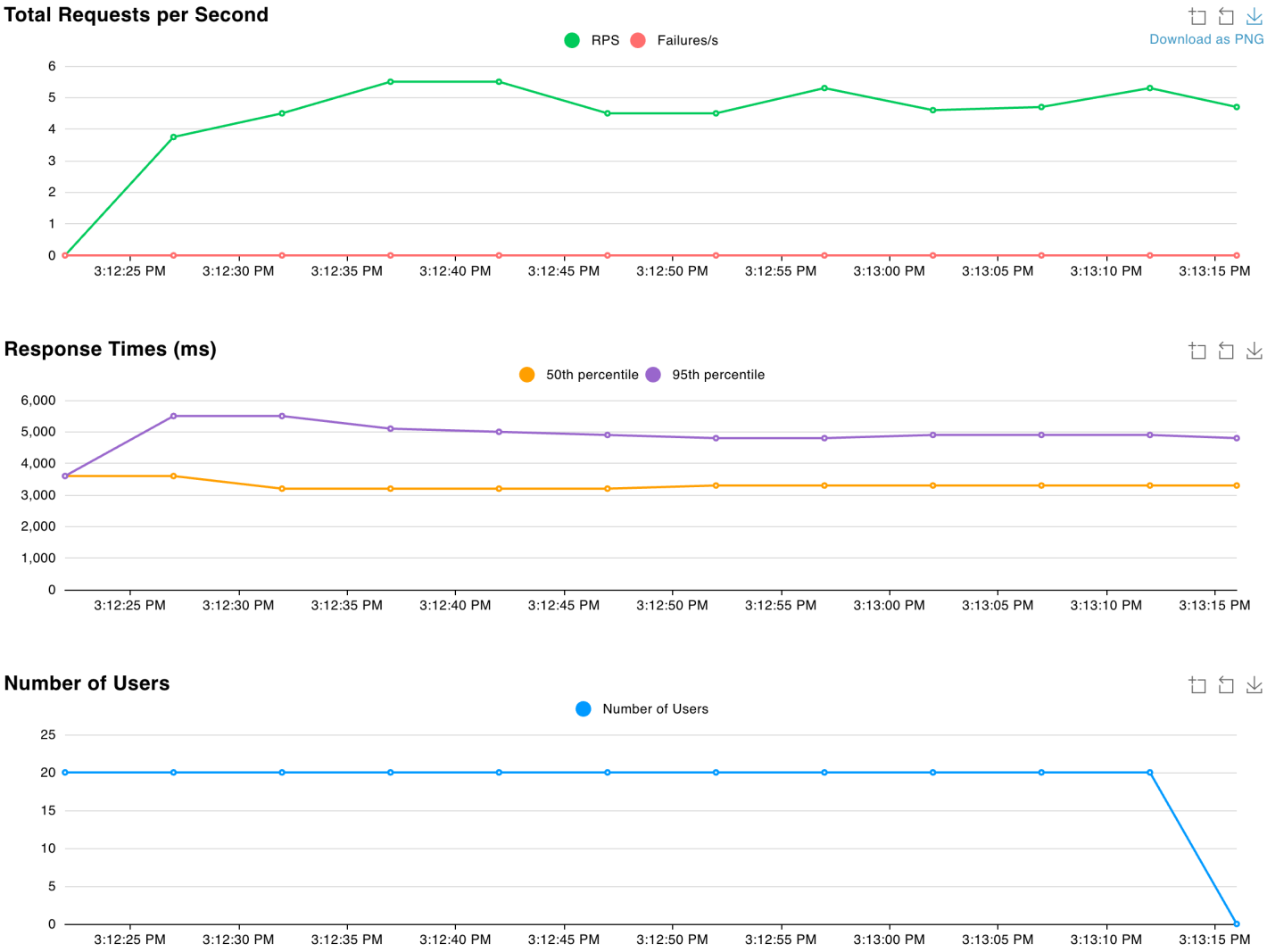

Figure: Sync Flash Sale - latency spikes as 20 users overwhelm 15 payment slots

Phase 3: Async Flash Sale - The Solution

Test config: 20 concurrent users, 60 seconds, spawn rate 10 users/sec, 1 worker goroutine

Metric	Value
Total Requests	2,727
Failure Rate	0%
Avg Response Time	132 ms
Median	110 ms
P99	520 ms
Throughput	45.70 req/s

By decoupling order acceptance from payment processing, the API now responds in ~132ms instead of 3,720ms. The system accepted 2,727 orders vs 285 in sync mode - a 9.6x improvement. Customers get an instant acknowledgment while payment is processed in the background.

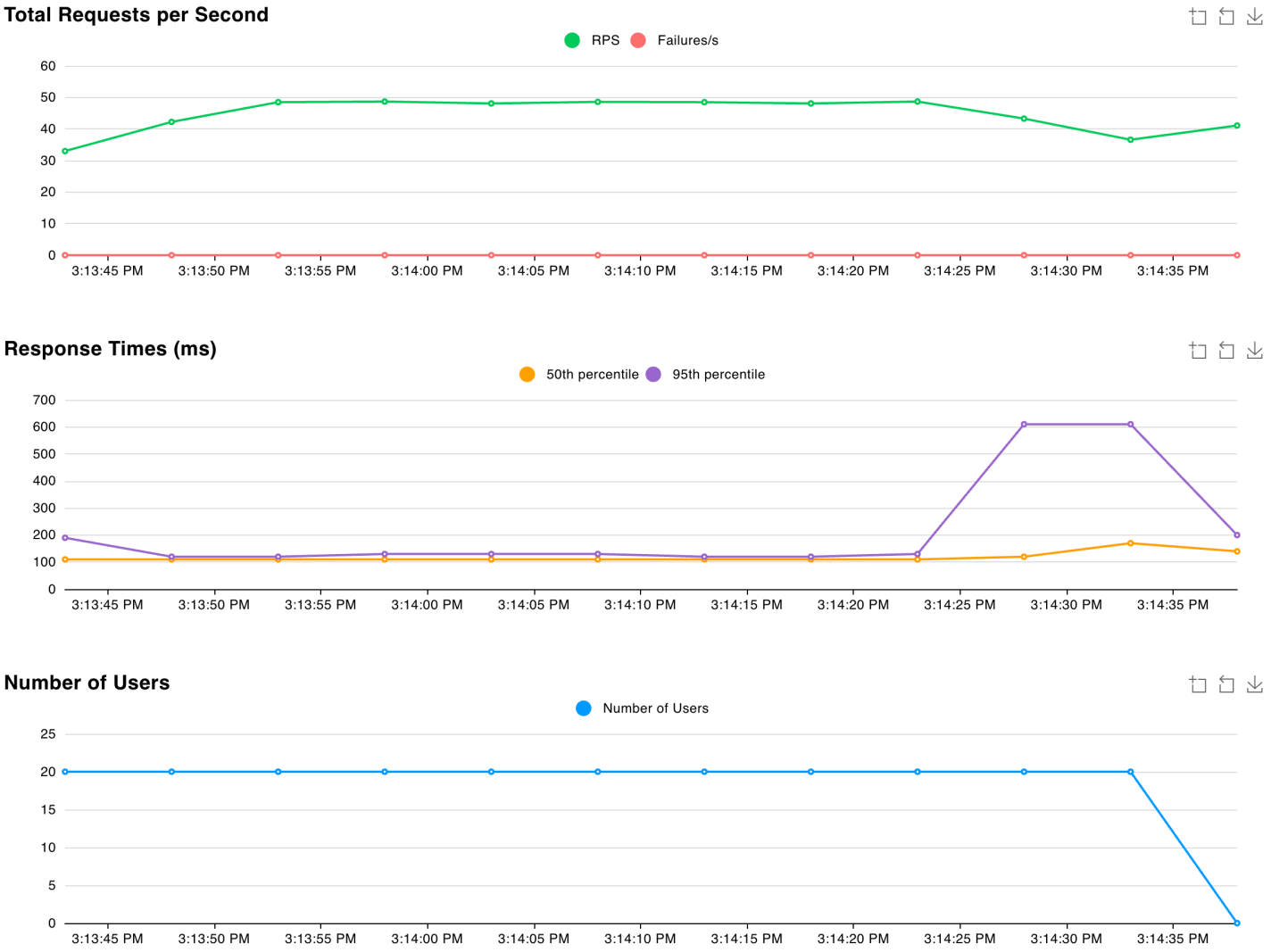

Figure: Async Flash Sale - consistent ~130ms response time with 20 users

Phase 4: Queue Analysis

While the async API accepts orders instantly, the single background worker can only process 0.33 orders/second (1 order per 3 seconds). During the 60-second test:

Metric	Value

Order acceptance rate	~45.7 orders/sec
Single worker rate	0.33 orders/sec
Queue growth rate	~45.4 messages/sec
Total messages queued	~2,727
Backlog clear time	~136 minutes

The SQS metric ApproximateNumberOfMessagesVisible in CloudWatch confirmed a rapid spike to ~5,260 messages during the test period, with a gradual drain as workers processed the backlog.

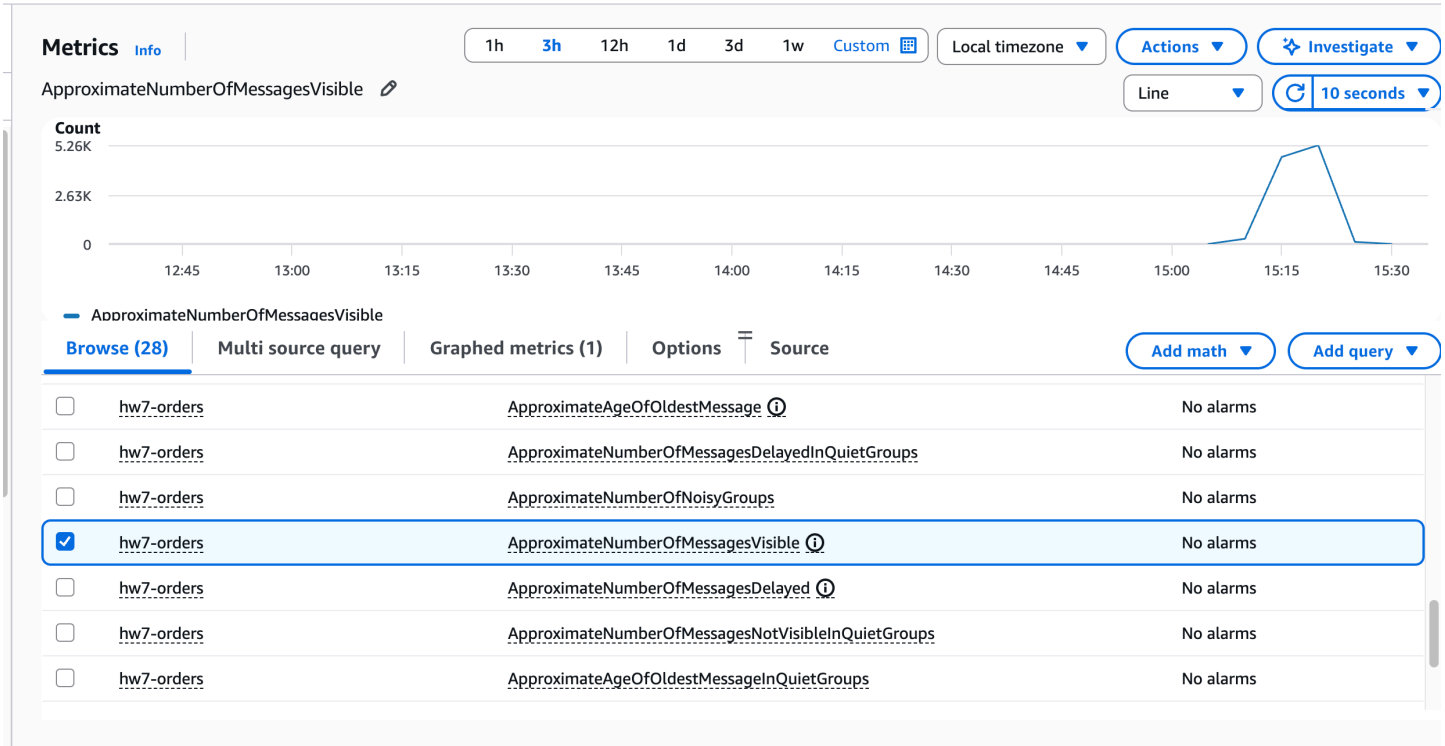

Figure 1: CloudWatch SQS ApproximateNumberOfMessagesVisible during flash sale tests

Phase 5: Worker Scaling

All tests used the same flash sale config: 20 users, 60 seconds, 10 users/sec spawn rate. The WORKER_COUNT environment variable was changed via Terraform to scale goroutines within a single ECS task.

Workers	Rate (ord/s)	Reqs	Avg (ms)	P99 (ms)	Clear Time
1	0.33	2,727	132	520	~136 min
5	1.67	2,880	111	180	~29 min
20	6.67	2,832	118	220	~7 min
100	33.33	2,829	113	190	~1.4 min

API response times remain consistent across all worker counts since the API only publishes to SNS. The key difference is how fast the queue drains. With 100 workers, the backlog clears in about 1.4 minutes after the flash sale ends.

Minimum workers to prevent queue buildup: the flash sale produces ~47 orders/second. Each worker handles 1 order per 3 seconds (0.33/sec). To match incoming rate: 47 x 3 = 141 workers would be needed.

Analysis Questions

How many times more orders did async accept vs sync?

The async endpoint accepted 2,727 orders vs 285 for sync in the same 60-second window - 9.6x more orders. Throughput went from 4.85 req/s to 45.70 req/s.

What causes queue buildup and how do you prevent it?

Queue buildup occurs when the rate of incoming messages exceeds the rate at which workers can process them. With a 3-second payment delay, each worker processes 0.33 orders/second. Prevention: scale goroutines within a task, scale ECS task count, or use auto-scaling policies tied to the ApproximateNumberOfMessagesVisible CloudWatch metric.

When would you choose sync vs async in production?

Sync: when the operation is fast (sub-second), the client needs an immediate result (e.g., login), or data consistency must be confirmed before responding.
Async: when the operation is slow or unpredictable (like payment verification), the system must stay responsive under load spikes, or eventual consistency is acceptable.

Part III: Lambda - What If You Didn't Need Queues?

Lambda Configuration

Setting	Value
Function Name	hw7-order-platform-processor
Runtime	provided.al2 (Go custom runtime)
Architecture	x86_64
Memory	512 MB
Timeout	30 seconds
Trigger	SNS topic hw7-orders (direct)
Processing	Same 3-second payment delay

Architecture simplification:

```
Part II:  Order API -> SNS -> SQS -> ECS Workers (you manage everything)
Part III: Order API -> SNS -> Lambda (AWS manages everything)
```

Test: Manual Orders via curl

10 orders were sent manually through the existing ALB async endpoint with 2-second spacing. All 10 returned 202 Accepted and were processed successfully by the Lambda function.

Cold Start Observations

From CloudWatch Logs (/aws/lambda/hw7-order-platform-processor):

Cold start (with Init Duration):

```
INIT_START Runtime Version: provided:al2.v143
REPORT RequestId: 55f9eb89...
  Duration: 3004.28 ms  Billed: 3075 ms
  Memory: 512 MB  Max Memory: 19 MB  Init Duration: 70.26 ms
```

Warm start (no Init Duration):

```
REPORT RequestId: 86511de3...
  Duration: 3001.37 ms  Billed: 3002 ms
  Memory: 512 MB  Max Memory: 20 MB
```

Metric	Cold Start	Warm Start
Init Duration	70-75 ms	N/A
Billed Duration	~3,075 ms	~3,004 ms
Overhead	~75 ms (2.5%)	0 ms
Memory Used	19-20 MB	20 MB

Cold starts occurred in 2 out of 10 invocations - the first request and when Lambda provisioned a second concurrent execution environment. A 75ms cold start on a 3,000ms payment operation adds only 2.5% overhead, which is negligible.

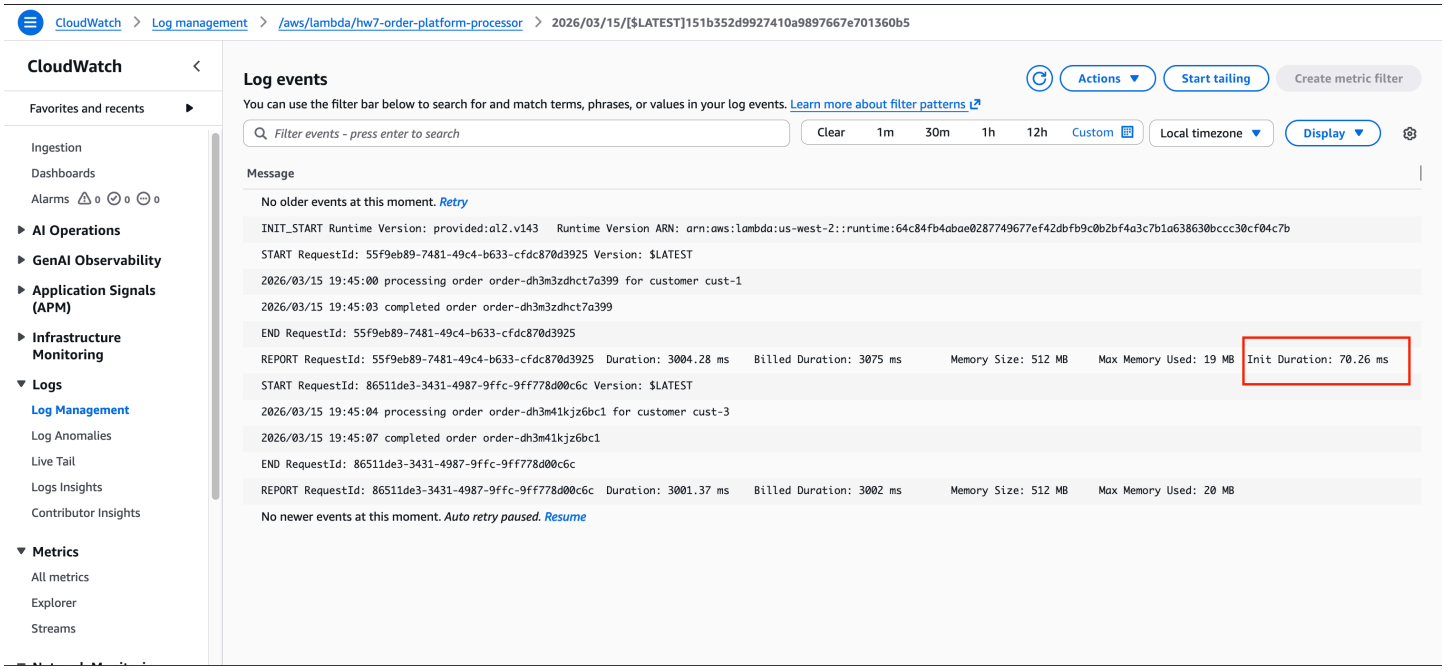

Figure 2: CloudWatch Lambda logs showing cold start (Init Duration) vs warm start

Cost Comparison

ECS cost (Part II): 2 Fargate tasks running 24/7 at 256 CPU / 512 MB = ~$17/month

Lambda cost for 10,000 orders/month:

Component	Calculation	Cost
Requests	10,000 (under 1M free tier)	$0
GB-seconds	10,000 x 3s x 0.5GB = 15,000	$0
Total		$0 (FREE)

Break-even: Lambda's free tier covers 400K GB-seconds / 1.5 GB-sec per order = ~267K orders/month. You would need ~1.7M requests/month before Lambda matches the $17/month ECS cost.

Trade-off Analysis

	ECS + SQS (Part II)	Lambda (Part III)
Cost (10K/mo)	$17/month	$0 (free tier)
Ops Overhead	High	Zero
Scaling	Manual	Automatic
Durability	SQS retains 4 days	SNS retries 2x only
Retry Control	Full (DLQ, visibility)	Limited
Cold Start	N/A (always on)	~75ms (2.5%)

Recommendation: Should the Startup Switch to Lambda?

For a startup processing under 267K orders/month, Lambda is the clear winner. It costs $0 compared to $17/month for ECS, requires zero operational overhead (no 3am queue depth alerts, no manual worker scaling, no health monitoring), and the 2.5% cold start penalty is negligible on 3-second payment processing. The main concern is the loss of SQS durability - if Lambda fails or SNS discards a message after 2 retries, that order is lost with no recovery path. For a cost-conscious startup with moderate volume, the trade-off is acceptable. As the business grows and order volume or reliability requirements increase, a hybrid approach (SNS -> SQS -> Lambda) would provide both the auto-scaling of serverless and the durability of queued processing.